Self-attention

Palabra 1 "duerme"

Palabra Q ¿Qué busco? K ¿Qué ofrezco? V mi información.

El	[1, 0]	[1, 0]	[1, 0]
gato	[1, 2]	[0, 2]	[0, 2]
duerme	[0, 2]	[1, 1]	[1, 1]

Real → miles de dimensiones y los valores los aprende la red.

Producto punto entre Q y K para cada palabra:

El	$0 \cdot 1 + 2 \cdot 0$	=	0
gato	$0 \cdot 0 + 2 \cdot 2$	=	4
duerme	$0 \cdot 1 + 2 \cdot 1$	=	2

Escalar: $\sqrt{d_k} = \sqrt{\text{tamaño}} = \sqrt{2} = 1,41$

Puntaje escalado: 0 / 2,83 / 1,41

Softmax:

$e^{0} = 1,00$
$e^{2,83} = 16,92$
$e^{1,41} = 4,11$
} $\Sigma(t) = 22,03$

1 / 22,03 = 4,5% → El
16,92 / 22,03 = 76,8% → gato
4,11 / 22,03 = 18,7% → duerme

Mezclar los values (V) con esos porcentajes

$0,045 \cdot [1,0] + \boxed{0,768 \cdot [0,2]} + 0,187 [1,1] = [0,23; 1,72]$

Interpretación: El nuevo vector de "duerme" ahora está compuesto en un 77% por información de "gato". La palabra entendió quién es el que duerme.

Ahora con palabra 2 "gato" → Q [1,2]
con 3 "El" → Q [1,0]

Resultado final:

Palabra	Vector antes	Vector con contexto
El	[1,0]	[0,80 : 0,80]
gato	[0,1]	[0,38 : 1,65]
duerme	[1,1]	[0,23 : 1,72]

fórmula: $\text{Atención}(Q, K, V) = \text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) V$

$Q \cdot K^T$ =

	El	gato	duerme
El	40%	20%	40%
gato	7%	62%	31%
duerme	5%	77%	19%

Matriz de atención
Cada palabra contra cada palabra.

¿1000 tokens? → matriz 1000 x 1000
un millón de puntajes
repetida en decenas de capas
y en varias cabezas.

→ Anotaciones importantes: Self-attention es un mecanismo de varios, dentro del Transformer.

→ Analizan tokens (pedazos de palabra).

→ La CPU también tiene paralelismo, pero poco: 8 a 64 núcleos. La GPU tiene miles.

→ Google construyó un chip propio, la TPU.
Quien adaptó la GPU fue NVIDIA.

→ Brackwell es la GPU (arquitectura de 2024), no un componente que se le añadió. Grace es una CPU Arm que se une a ella.

NVIDIA. 2007: Lanza CUDA, para cálculo científico, antes de que la IA fuera el foco.

2009 - 2012: Investigadores descubren por su cuenta que las GPUs entrenan redes mucho más rápido. (AlexNet).

NVIDIA se especializa: CUDNN (2014), NV Link (2016),

Tensor Cores (2017) y luego Hopper y Blackwell.

Pesos: (w_Q, w_K, w_V) se ajustan con descenso por gr.
Capas: cada capa recibe los vectores de la anterior y los refina. Las 1eras captan relaciones simples y las otras más abstractas.

Cabeza: es una atención independiente dentro de una capa.

GPT-2: 12 capas x 12 cabezas = 144 mil veces.
con 1.000 tokens: 144 millones de puntajes.

En el código: 10 veces: Se repite toda la atención completa (las 4096 palabras juntas) 10 veces, y se promedia el tiempo.

```
Q = torch.randn(n, d, device = dev)
K = ...
V = ...
```

Cada repetición genera números aleatorios distintos, así que Q, K, V son diferentes.

Qué es dev: dispositivo donde se calcula y recibe el texto 'cpu' o 'cuda'.

synchronize() solo afecta el cronómetro.
El cronómetro se detendría de inmediato antes de que la GPU termine, ya que hace una operación, la deja y sigue con la siguiente. (en cola). En CPU no hace falta pq cada operación termina antes de pasar a la siguiente.

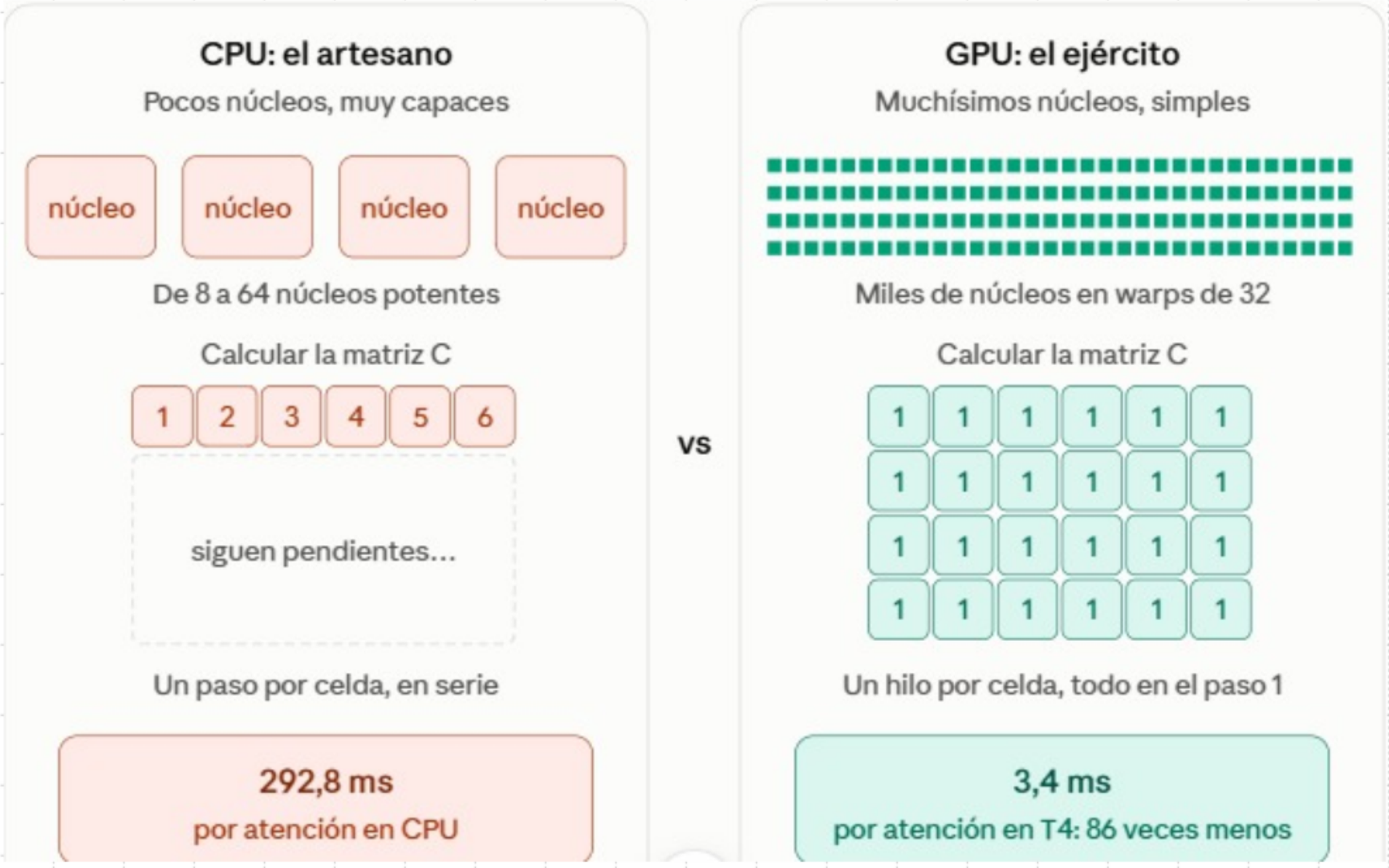
atención(Q, K, V) → Calentamiento 1era vez GPU (costos de arranque) no se quiere medir t en eso

t0 = time.perf_counter() → Inicia el cronómetro.

for _ in range(veces):
  atencion(Q,K,V) → 10 tablas de 4096 x 4096 ≈ 16,8 millones de puntajes

return (time.perf_counter() - t0) / veces → tiempo total / 10. el tiempo prom. de una atención en seg.

Respuesta en ms //



Hay que llenar la matriz C y cada celda es un calculo independiente.

CPU: el trabajo avanza celda por celda.

GPU: hay miles de núcleos simples en filas de 32 (los warps). Cada hilo toma una celda y todos quedan en el paso 1.

Los SM: la Tesla T4 tiene 40 SM ← los warps (32) se organizan en bloques de

Las GPUs de gama alta superan los 100 SM.

Esquema completo:

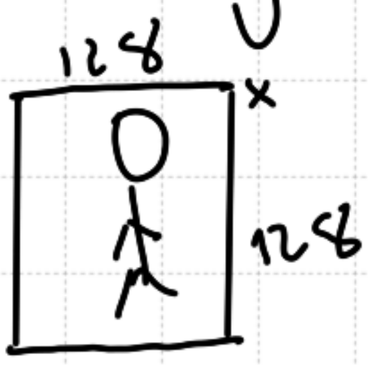
GPU (40 SM en la T4)
- SM (el taller)
  - Bloque (un encargo asignado al taller)
    - Warp (escuadra de 32)
      - Hilo (calcula 1 celda de C).

Para las 4096x4096 → 16.777.216 hilos para

focus = 2 ej: Sub-marino.

16/10/26

Pooling = Subsampling



kernel 3x3 → 32 → maxpooling ⇒ 64 x64 → kernel 3x3 32

→ maxpooling ⇒ 32 x32 → profundidad 32

1 Kernel para cada imagen y de esos resultados se suman o promedian para dar 1 edo.

local

CPU → Intel i7 Ultra 2da gen

GPU → NVDIA RTX 5070

Mismo algoritmo 4096 tokens.